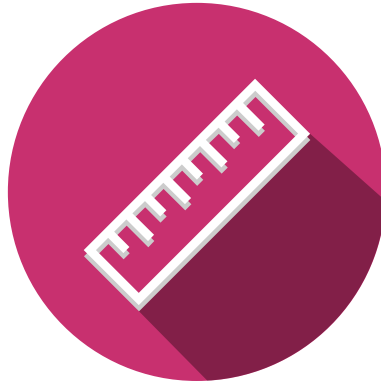




Cursor (Cur)

Course Conception Numérique (CNum)



Orientation: [Systèmes industriels \(Synd\)](#)

Spécialisation: [Infotronics \(IT\)](#)

Cours: [Conception Numérique \(CNum\)](#)

Auteurs: [Silvan Zahno](#), [Axel Amand](#), [François Corthay](#), [Christophe Bianchi](#)

Date: 09.09.2026

Version: v3.1.1



Contenu

1	Introduction	3
2	Spécification	4
2.1	Fonctions	4
2.2	Circuit	4
2.3	Scénario (exemple)	6
2.4	Projet HDL-Designer	6
3	Composants	7
3.1	Chariot	7
3.2	Circuit de commande de moteur	7
3.2.1	Moteur à courant continu	8
3.3	Codeur (encodeur)	9
3.4	Reed-Relais	10
3.5	Carte FPGA	10
3.6	Boutons et LEDs	11
4	Evaluation	12
5	Premières étapes	13
5.1	Tips	13
	Glossaire	14



1 Introduction

L'objectif de ce projet est d'appliquer concrètement les connaissances acquises à travers un exemple pratique en fin de semestre. Il consiste à contrôler un moteur à courant continu afin de déplacer précisément un chariot le long d'une vis vers des positions prédéfinies. Ce système de positionnement peut être observé dans la [Figure 1](#).

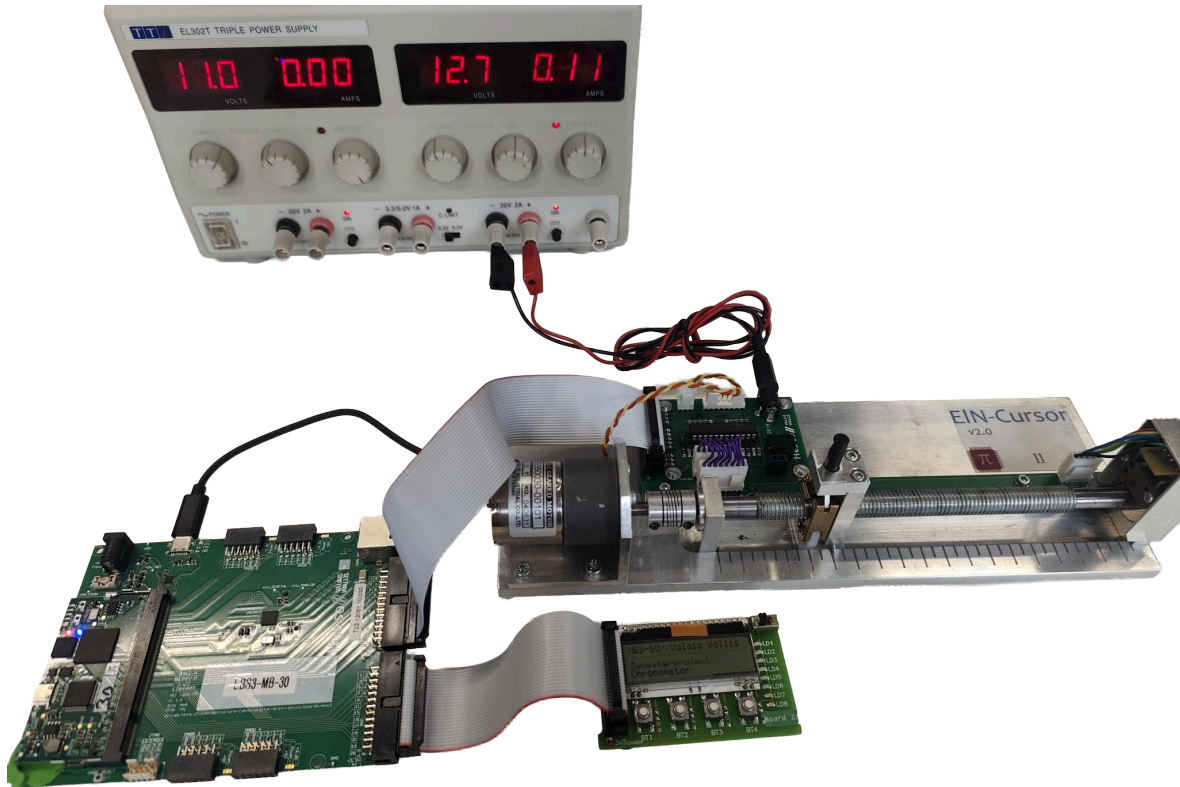


Figure 1 - Équipement du curseur (EBS3)

Les spécifications minimales (voir [Chapitre 2](#)) doivent être respectées, mais les étudiants sont encouragés à implémenter des fonctionnalités supplémentaires. Par exemple, un écran [Liquid Crystal Display \(LCD\)](#) peut être utilisé pour afficher diverses informations.



L'implémentation de fonctionnalités supplémentaires permettra d'obtenir des points supplémentaires dans l'évaluation finale.



2 | Spécification

2.1 Fonctions

Les fonctions de base sont définies comme suit :

- Lorsque la touche **restart** est appuyée, le curseur se déplace vers la position de départ indiquée par un relais reed ([Chapitre 3.4](#)) situé à proximité du moteur à courant continu ([Chapitre 3.2.1](#)).
- Lorsque l'on appuie sur la touche Position₁, le curseur doit d'abord accélérer régulièrement vers la position 1 (p_1), puis avancer à pleine vitesse et enfin ralentir régulièrement pour s'arrêter à la position 1 (p_1). Cela peut se faire à partir de la position de départ ou de la position 2, voir [Figure 2](#).
- Lorsque l'on appuie sur la touche Position₂, le curseur doit d'abord accélérer régulièrement vers la position 2 (p_2), puis avancer à pleine vitesse et enfin ralentir régulièrement pour s'arrêter à la position 2 (p_2), voir [Figure 2](#).

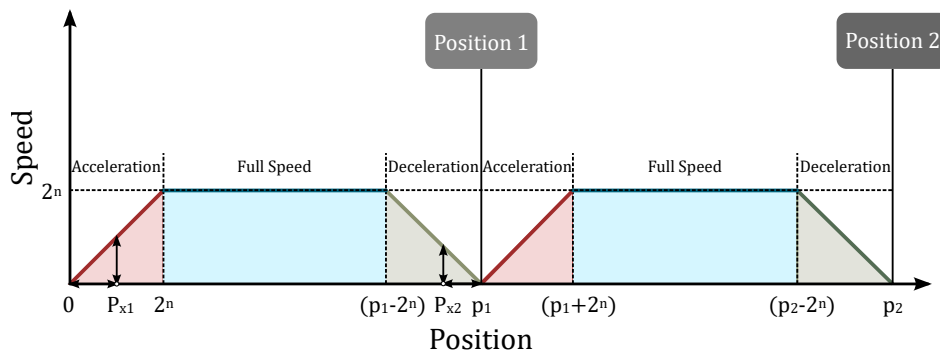


Figure 2 - Diagramme de la vitesse du chariot

Les positions à atteindre sont :

- Position 0 (p_0 , position de réinitialisation) = 3.3 - 3.5 cm suivant les assemblages mécaniques
- Position 1 (p_1) = 8 cm
- Position 2 (p_2) = 12 cm

2.2 Circuit

Pour accomplir la tâche, le circuit suivant est donné pour déplacer le chariot.

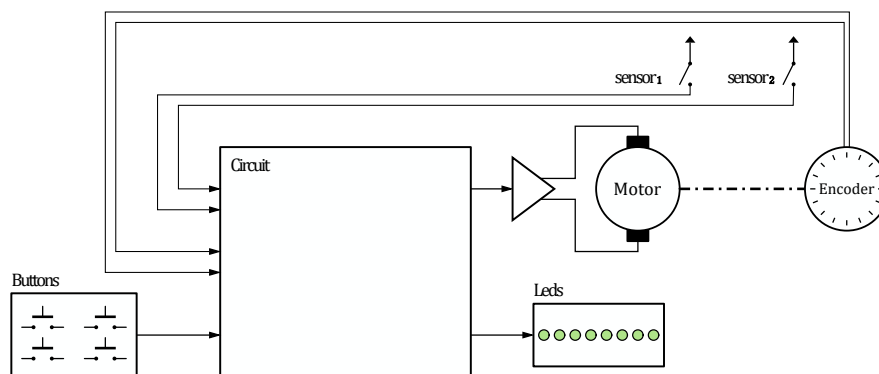


Figure 3 - Circuit du curseur



Le circuit fonctionne comme suit :

- Le moteur à courant continu ([Chapitre 3.2.1](#)) est commandé par les trois signaux motor_{On} , side_1 , side_2 :
 - Le moteur est activé lorsque le signal motor_{On} est à 1 et doit être remis à 0 lorsque le moteur ne doit pas tourner.
 - Sa vitesse est contrôlée par une modulation **Pulse Width Modulation (PWM)** appliquée soit au signal side_1 , soit au signal side_2 selon le sens de rotation désiré.
- Deux relais Reed ([Chapitre 3.4](#)) sensor_1 et sensor_2 sont placés aux extrémités du rail [1]. Ils détectent la présence du chariot du curseur en indiquant la présence d'un aimant par un 1.
- L'encodeur ([Chapitre 3.3](#)) est utilisé pour suivre, respectivement compter la position du curseur. Ses trois sorties, encoder_A , encoder_B et encoder_I , permettent de suivre les mouvements de la vis. Les sorties A et B alternent entre 0 et 1 lors des déplacements avec un déphasage de 90° entre elles, ce qui permet de déterminer la direction du mouvement. La sortie I génère une impulsion 1 à chaque tour complet de la vis.
- Trois touches sont utilisées pour contrôler le système: restart , go_1 et go_2 . Une touche supplémentaire, button_4 , peut être utilisée pour des fonctions optionnelles. Lorsque l'une des touches est enfoncées, le signal correspondant passe à 1.
- Les broches testOut peuvent être utilisées pour sortir des informations supplémentaires du système, notamment pour les brancher sur les **Light Emitting Diodes (LEDs)** à des fins de debug ou visualisation
- Le signal testMode est mis à 1 lors de la simulation. Ce dernier peut être utilisé pour raccourcir les compteurs utilisés afin d'accélérer la génération de signaux lors des tests.

Le toplevel vide du design ([cursor-toplevel-empty.pdf](#)) montre tous les signaux connectés à la platine **Field Programmable Gate Array (FPGA)** Figure 4.



Figure 4 - Circuit Toplevel vide



2.3 Scénario (exemple)

Dans le [Figure 5](#), trois scénarios différents sont présentés. Tout d'abord, on appuie sur la touche restart et le chariot se déplace à pleine vitesse vers la position initiale (sensor₁). Les deux autres scénarios go₂ et go₁ déplacent le chariot vers la position₂ et la position₁ respectivement, un signal PWM variable est appliqué aux signaux side₂ et side₁ pour accélérer et ralentir le chariot.

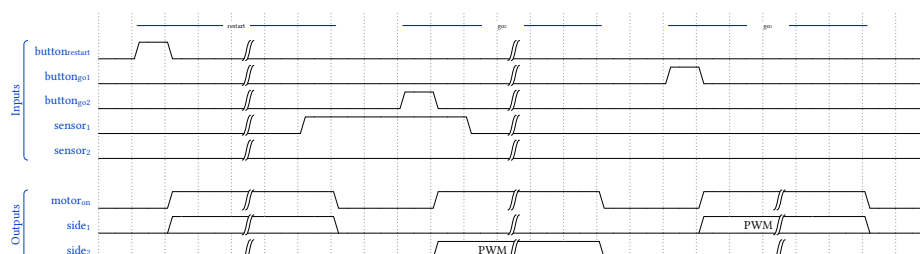


Figure 5 - Scénario curseur



Les scénarios ci-dessus sont des exemples. C'est aux étudiants de les compléter.

2.4 Projet HDL-Designer

Un projet HDL-Designer prédéfini peut être téléchargé ou cloné dans [Cyberlearn](#) ou [Github](#). La structure de fichier du projet se présente comme suit:

```
did_cursor
+--Board/           # Project and files for programming the FPGA
|   +--concat/      # Complete VHDL file including PIN-UCF file
|   +--ise/         # Xilinx ISE project
+--Cursor/          # Library for the components of the student solution
+--Cursor_test/     # Library for the simulation testbenches
+--doc/             # Folder with additional documents relevant to the project
|   +--Board/       # All schematics of the hardware boards
|   +--Components/  # All data sheets of hardware components
+--img/             # Pictures
+--Libs/            # External libraries which can be used e.g. gates, io, sequential
+--Prefs/           # HDL-Designer settings
+--Scripts/         # HDL-Designer scripts
+--Simulation/      # Modelsim simulation files
```



Le chemin d'accès au dossier du projet ne doit pas contenir d'espaces.



Le dossier **doc/** contient de nombreuses informations importantes: fiches techniques, évaluation de projet et documents d'aide pour HDL-Designer, pour n'en citer que quelques-unes.



3 Composants

Le système se compose de 3 platines matérielles différentes, visibles dans la [Figure 1](#).

- Un assemblage de chariot avec une carte électronique **Printed Circuit Board (PCB)** qui commande le moteur et lit les capteurs ([Figure 6](#))
- Une carte de développement **FPGA** ([Figure 14](#))
- Une carte de contrôle à 4 boutons et 8 **LEDs** ([Figure 15](#))

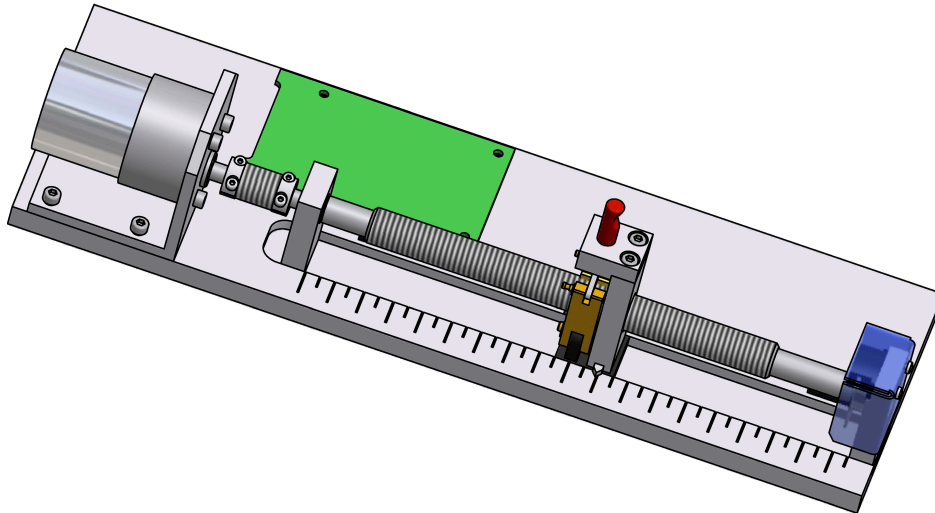


Figure 6 - Assemblage de chariot du curseur

3.1 Chariot

La structure du chariot comprend le moteur à courant continu, les deux relais reed [Chapitre 3.4](#) ainsi que le chariot et la vis. Le pas de la vis est de M12x1.75, ce qui signifie qu'une distance de 1.75mm est parcourue par révolution ([Figure 7](#)).

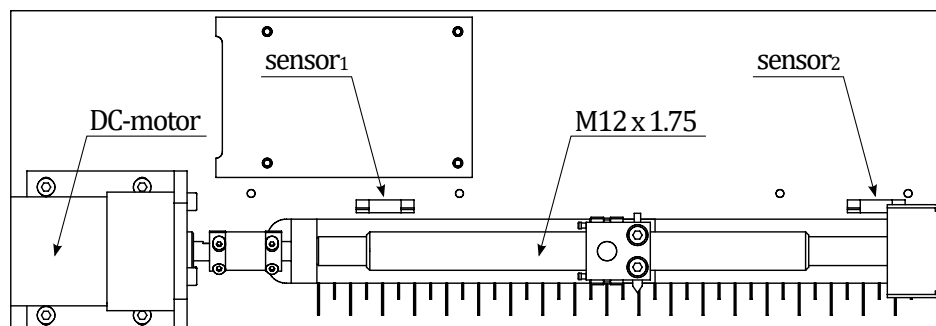


Figure 7 - Assemblage détaillé du chariot du curseur

3.2 Circuit de commande de moteur

Le moteur à courant continu du chariot est alimenté en 12V. La carte d'alimentation possède un pont en H commandé par des signaux numériques. Sur la platine d'alimentation, un régulateur 5V génère la tension alimentant la platine [FPGA](#) [2].



3.2.1 Moteur à courant continu

Le moteur à courant continu est commandé par un driver de pont en H L6207 [3], voir figure Figure 8. La fréquence de commutation maximale du pont en H est de 100kHz. Ceci doit être pris en compte lors de la création du signal PWM.

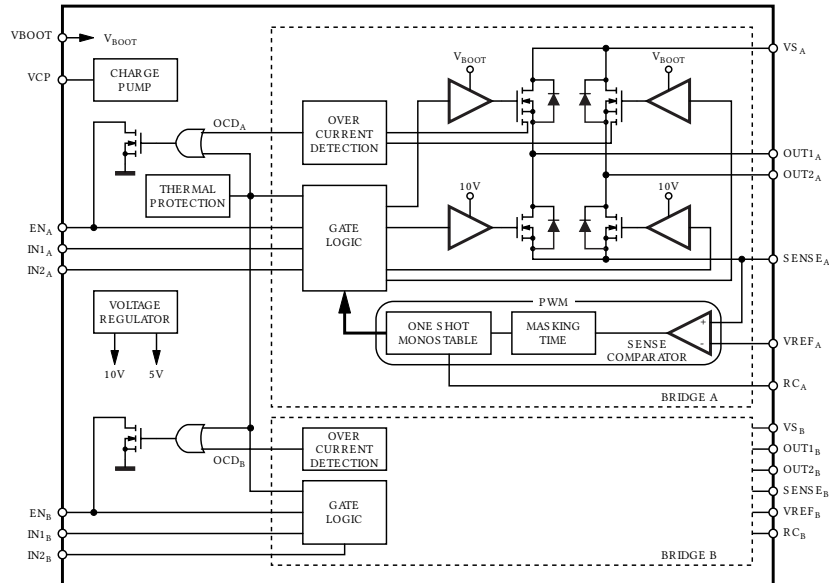


Figure 8 - Schéma bloc du circuit du pont-H L6207N [3]

Pour ajuster la vitesse du moteur à courant continu, un signal PWM doit être appliqué aux signaux side₁ ou side₂, la fréquence maximale étant de 100kHz. Plus la tension est appliquée longtemps au moteur, plus il tourne vite. Alimenter soit side₁ ou side₂ contrôle le sens du moteur.

PWM - Stratégie d'implémentation

Une stratégie consiste à créer un compteur en dents de scie ($0, 1, 2 \dots 2^N - 2, 2^N - 1, 0, 1, \dots$) et à comparer sa valeur à un seuil. La règle est de sortir "1" lorsque la valeur du compteur est inférieure au seuil, et "0" sinon. En le modifiant, le rapport cyclique du signal PWM est modifié. De cette façon, la fréquence du signal PWM est définie par la vitesse du compteur (dents de scie), et le rapport cyclique est défini par la valeur du seuil.

Dans la Figure 9, le moteur tourne plus lentement avec le signal vert qu'avec le signal bleu et qu'avec le signal rouge.

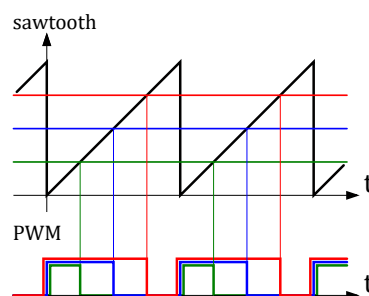


Figure 9 - Signaux PWM



Plus haut est le nombre de pas (bits) de la **PWM**, plus le contrôle de la vitesse est précis. Cependant, trop de pas peuvent entraîner une différence de vitesse non notable en raison des systèmes de contrôle et des caractéristiques du moteur. Une bonne base est d'utiliser 8 bits pour la résolution du **PWM**, donnant 256 niveaux de vitesse de 0 à 255.

Rampes - Stratégie d'implémentation

Pour créer des rampes d'accélération et de décélération, une méthode courante consiste à utiliser un profil de vitesse trapézoïdal. Ce profil se compose de trois phases : accélération, vitesse constante et décélération, comme le montre la **Figure 10**.

Plusieurs stratégies existent pour créer de telles rampes. Typiquement, elles sont basées sur le temps. Mais un tel système nécessite une boucle de rétroaction aussi longue qu'une boucle de contrôle de position.

Une méthode plus simple à mettre en œuvre consiste à baser les rampes sur la position selon l'équation $v_x = m * x + b$.

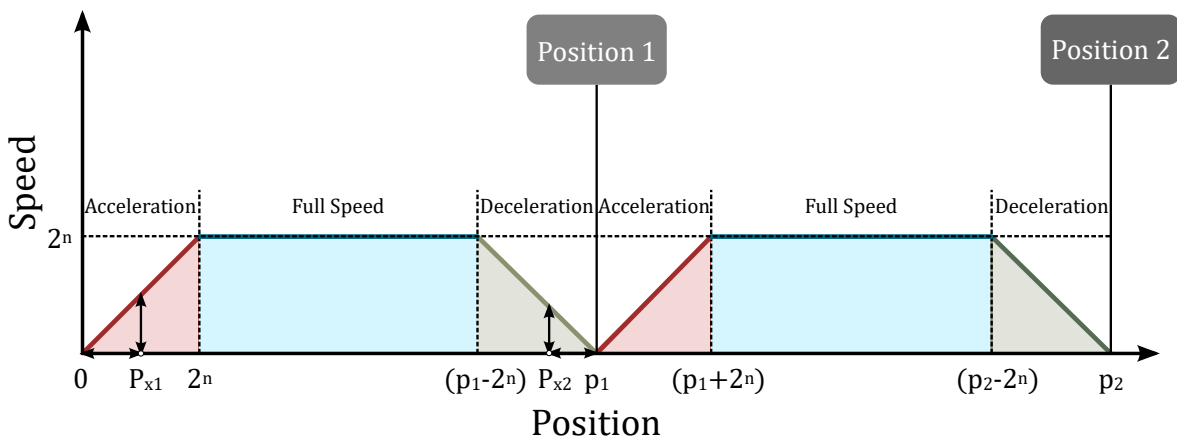


Figure 10 - Diagramme de la vitesse du chariot

3.3 Codeur (encodeur)

L'angle de la vis peut être mesuré à l'aide d'un **codeur incrémental**. Le modèle utilisé dans l'assemblage est un AEDB-9140-A12 [4] (**Figure 11**) avec 2 canaux de feedback avec 500 **Counts per Revolution (CPR)** (impulsions par tour) par canal, représenté dans la **Figure 12**.

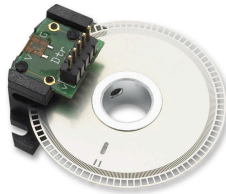


Figure 11 - Encodeur AEDB-9140-A12 [4]

Encodeurs - Stratégie d'implémentation

Il est possible de l'utiliser de deux manières:

- En utilisant un seul canal, ce qui donne une résolution de $500 \frac{\text{pulses}}{\text{tour}}$, c.à.d $1'000 \frac{\text{fronts}}{\text{tour}}$ si l'on compte les fronts montants et descendants.
- En utilisant les deux canaux comme interface **QEI (Quadrature Encoder Interface)**, en comptant les fronts montants et descendants des deux canaux, augmentant la résolution à $2000 \frac{\text{fronts}}{\text{tour}}$. Cet



interfaçage permet aussi de détecter le sens de rotation du moteur et est une méthode courante pour contrôler les moteurs à courant continu avec des encodeurs de feedback dans l'industrie.

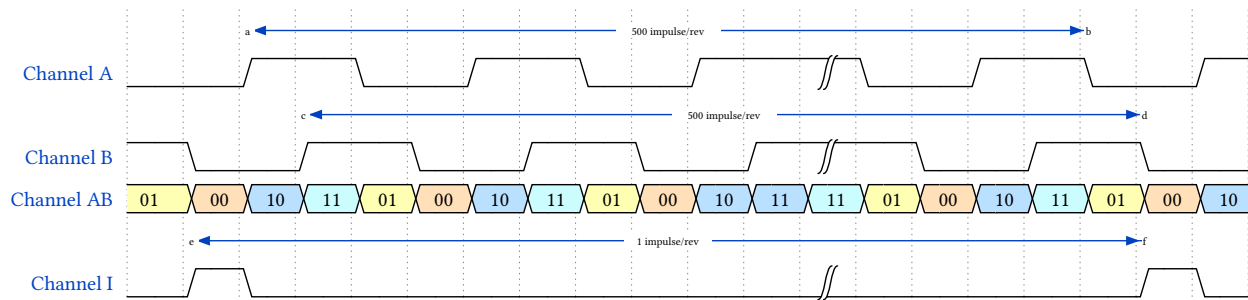


Figure 12 - Signaux de codeurs incrémentaux

3.4 Reed-Relais

Le relais Reed est un interrupteur qui peut être commuté à l'aide d'électro-aimants [1]. Lorsqu'un aimant se trouve à proximité du capteur, le contact se ferme (Figure 13). 2 relais Reed sont utilisés (sensor_1 et sensor_2) pour identifier les limites gauche et droite du chariot.

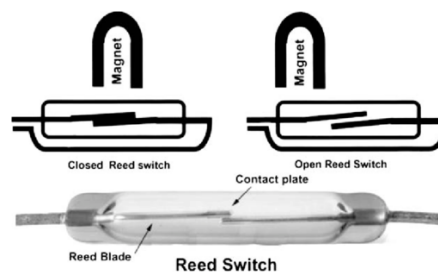


Figure 13 - Reed relais [5]

Les signaux sensor_1 et sensor_2 sont à "1" lorsque le contact est fermé (aimant à proximité), sinon "0".

3.5 Carte FPGA

La carte principale est la carte de développement de laboratoire FPGA-EBS3 de l'école [6]. Elle héberge une puce **FPGA Lattice LFE5U-25F** et dispose de nombreuses interfaces différentes (**Universal Asynchronous Receiver Transmitter (UART)**, **Universal Serial Bus (USB)**, Ethernet, etc.). L'oscillateur utilisé produit un signal d'horloge (**clock**) avec une fréquence de $f_{\text{clk}} = 100\text{MHz}$, réduit par PLL à $f_{\text{clk}} = 60\text{MHz}$.

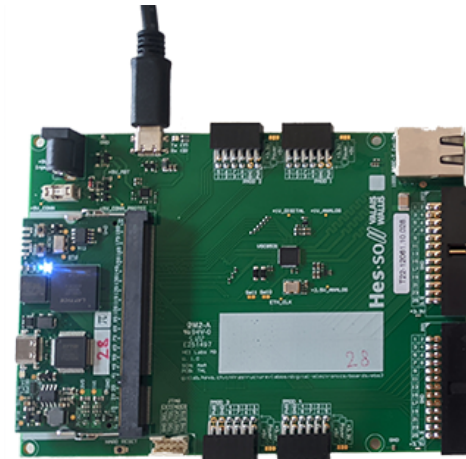


Figure 14 - Carte électronique FPGA EBS3 [6]

3.6 Boutons et LEDs

La platine avec les boutons et les LEDs [7] est connectée à la platine FPGA. Elle comprend 4 boutons et 8 LEDs qui peuvent être utilisés dans le design. Cette platine comprend un affichage LCD [8], [9] qui peut être utilisé dans le projet.

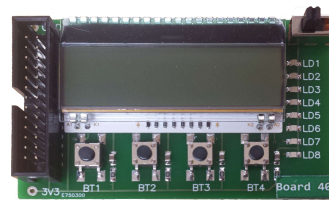


Figure 15 - Carte électronique boutons-LEDs-LCD [7]



4 | Evaluation

Dans le dossier **doc/**, le fichier **evaluation-bewertung-cursor.pdf** montre le schéma d'évaluation détaillé, voir [Table 1](#).

La note finale contient le rapport, le code ainsi qu'une présentation de votre système.

Aspects évalués	Points
Rapport	55
Introduction	3
Spécification	5
Projet	20
Vérification et validation	10
Intégration	9
Conclusion	3
Aspects formels du rapport	5
Fonctionnalité du circuit	30
Qualité de la solution	10
Présentation	10
Total	105

Table 1 - Grille d'évaluation



La grille d'évaluation donne des indications sur la structure du rapport. Pour un bon rapport, consultez le document « Comment rédiger un rapport de projet » [\[10\]](#).



5 | Premières étapes

Pour commencer le projet, procédez de la manière suivante :

- Lisez attentivement les spécifications et les informations ci-dessus.
- Examinez le matériel et testez le programme préprogrammé.
- Parcourez les documents dans le dossier **doc/** de votre projet.
- Développez un schéma fonctionnel détaillé. Vous devez pouvoir expliquer les signaux et leurs fonctions.
- Implémentez et simulez les différents blocs.
- Testez la solution sur le circuit imprimé et trouvez les éventuelles erreurs 🐛.

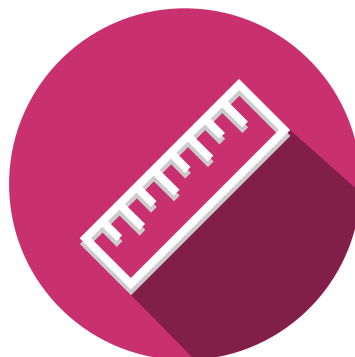
5.1 Tips

Ci-joint quelques conseils supplémentaires pour éviter les problèmes et les pertes de temps:

- Divisez le problème en différents blocs, utilisez pour cela le document Toplevel vide (**cursor-toplevel-empty.pdf**). Il est recommandé d'avoir un mélange équilibré entre le nombre de composants et la taille/complexité des composants.
- Analysez les différents signaux d'entrée et de sortie. Il est conseillé d'utiliser en partie les fiches techniques.
- Respectez le chapitre DiD « Méthodologie de conception de circuits numériques (MET) » lors de la création du système [11].
- Il est recommandé de réaliser le système de façon incrémentale, par exemple:
 - D'abord un système qui réagit aux boutons et qui amène le chariot à la position correspondante (toujours à la vitesse maximale).
 - Intégrer les phases d'accélération dans le système existant.



N'oubliez pas de vous amuser.





Glossaire

CPR – Counts per Revolution [9](#)

FPGA – Field Programmable Gate Array [5](#), [7](#), [10](#), [11](#)

LCD – Liquid Crystal Display [3](#), [11](#)

LED – Light Emitting Diode [5](#), [7](#), [11](#)

PCB – Printed Circuit Board [7](#)

PWM – Pulse Width Modulation [5](#), [6](#), [8](#), [9](#)

UART – Universal Asynchronous Receiver Transmitter [10](#)

USB – Universal Serial Bus [10](#)



Bibliographie

- [1] « Reed Relay », 5 décembre 2020. Consulté le: 24 novembre 2021. [En ligne]. Disponible sur: https://en.wikipedia.org/w/index.php?title=Reed_relay&oldid=992433034
- [2] Olivier Walpen, « Schematic: Cursor Chariot Power Circuit ». 2009.
- [3] STMicroelectronics, « Datasheet: DMOS Dual Full Bridge Driver with PWM Current Controller ». 2003.
- [4] Agilent Technologies, « Datasheet Agilent AEDB-9140 Series Three Channel Optical Incremental Encoder Modules with Codewheel, 100 CPR to 500 CPR ». 2005.
- [5] « Magnetic-Reed-Switch-Above-Closed-and-open-reed-switch-in-response-to-magnet-placement.Png (850×345) ». Consulté le: 24 novembre 2021. [En ligne]. Disponible sur: <https://www.researchgate.net/profile/Sidakpal-Panaich-2/publication/51169357/figure/fig1/AS:394204346896388@1470997048549/Magnetic-reed-switch-Above-Closed-and-open-reed-switch-in-response-to-magnet-placement.png>
- [6] A. Amand et S. Zahno, « FPGA-EBS3 Electornic Technical Documentation ». 2022.
- [7] Silvan Zahno, « Schematic: Parallelport HEB LCD V2 ». 2014.
- [8] Sitronix, « Datasheet Sitronix ST7565R 65x1232 Dot Matrix LCD Controller/Driver ». 2006.
- [9] Electronic Assembly, « Datasheet: DOGM Graphics Series 132x32 Dots ». 2005.
- [10] Christophe Bianchi, François Corthay, et Silvan Zahno, « Comment Rédiger Un Rapport de Projet? ». 2021.
- [11] François Corthay, Silvan Zahno, et Christophe Bianchi, « Méthodologie de Conception de Cicuits Numériques ». 2021.